

CSS6224 NETWORK SECURITY

An abstract digital network visualization. It features numerous glowing blue nodes of varying sizes, some with circular halos, interconnected by thin, light-blue lines. The lines form a complex web, with some bundles of lines entering from the left and branching out towards the right. The background is dark blue with soft, out-of-focus bokeh light spots.

Lecture 13: Application-Level Security



Overview

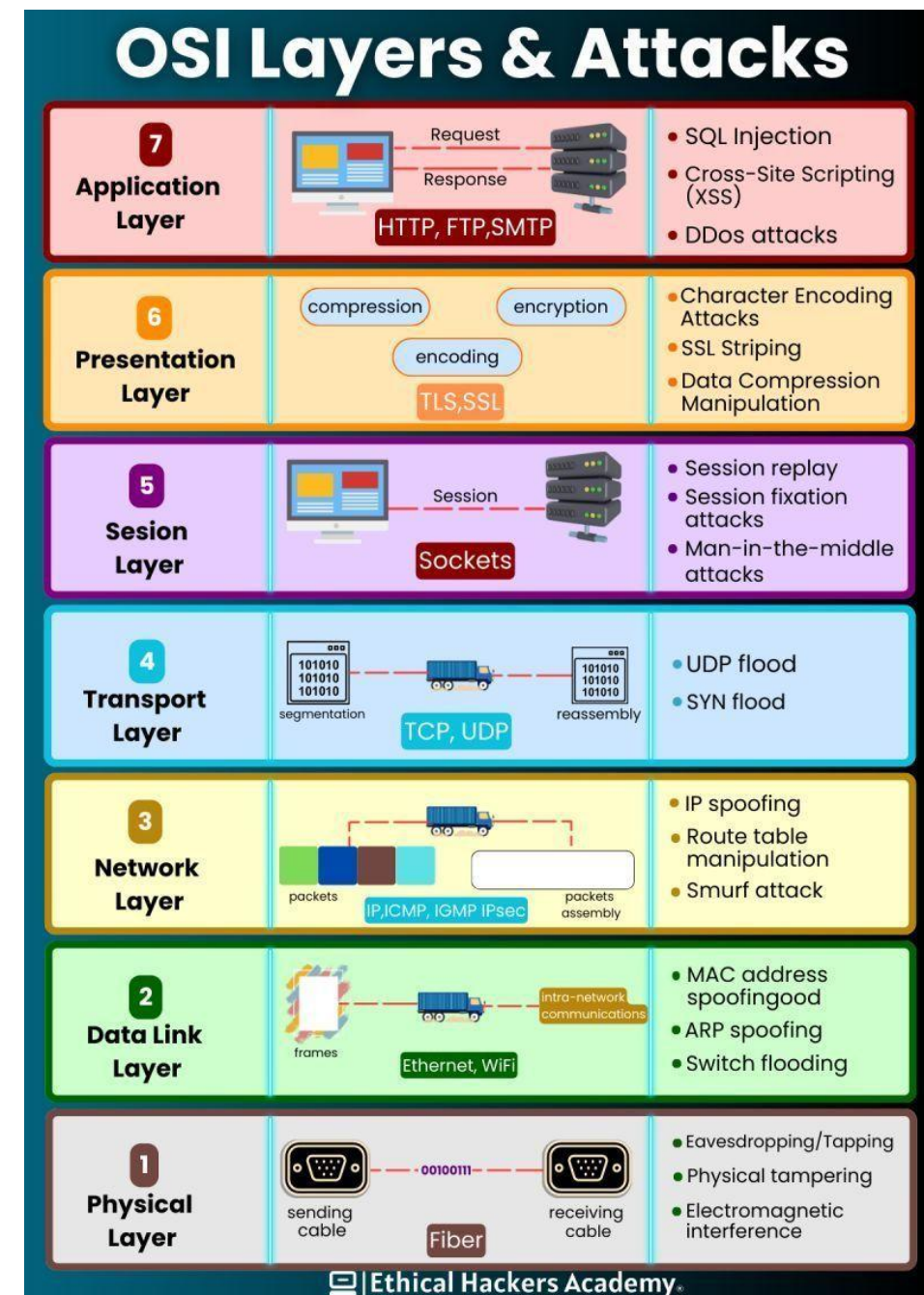
- **What is Application-Level Security?**
 - Protection of applications from malicious attacks that exploit vulnerabilities at Layer 7 (application layer) of the OSI model.
- **Why it Matters?**
 - 80–90% of modern cyberattacks target applications, not just networks.
- **Scope of Application Security:**
 - Covers web apps, mobile apps, APIs, email clients, and more.
- **Common Threats:**
 - SQL Injection
 - Cross-Site Scripting (XSS)
 - Cross-Site Request Forgery (CSRF)
 - Remote Code Execution (RCE)

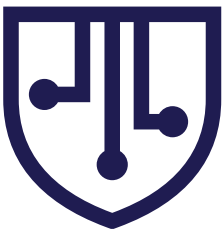
Defensive Tools and Mitigation Techniques:

- Input validation, session management
- Encryption, authentication mechanisms
- Application firewalls, secure coding practices

Application Security in the OSI Model

- **Application Layer (Layer 7):** Interface between end-user and network. Handles protocols like HTTP, FTP, SMTP, DNS.
- **Why Layer 7 is Targeted:**
 - Exposed to end-users and external networks
 - Complex, dynamic content (e.g., forms, sessions, cookies)
 - Human interaction increases risk of error and social engineering
- **Examples of Application-Level Services:**
 - Web Browsers
 - Email Clients
 - API Services
 - Remote Access Apps
- **Security Responsibility:** Must be enforced at the software level (developers) and network edge (firewalls, proxies)





Common Threats Targeting the Application Layer

1. SQL Injection (SQLi)

- Attackers insert malicious SQL queries into input fields
- Exploits poorly validated inputs to manipulate databases
- **Impact:** Unauthorized access, data leakage, data manipulation

2. Cross-Site Scripting (XSS)

- Injects malicious JavaScript into web pages
- Affects users who view the compromised page
- **Impact:** Session hijacking, credential theft

3. Cross-Site Request Forgery (CSRF)

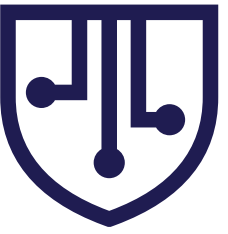
- Tricks users into executing unintended actions while authenticated
- Often disguised as a legitimate request (via email, image, etc.)
- **Impact:** Unwanted transactions, account misuse

4. Remote Code Execution (RCE)

- Attacker executes arbitrary code on the target server
- Can fully compromise web servers
- **Impact:** Full system takeover

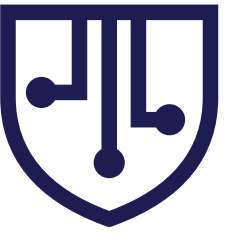
5. Broken Authentication & Session Management

- Weak login processes, predictable tokens, session fixation
- **Impact:** Account hijacking, privilege escalation



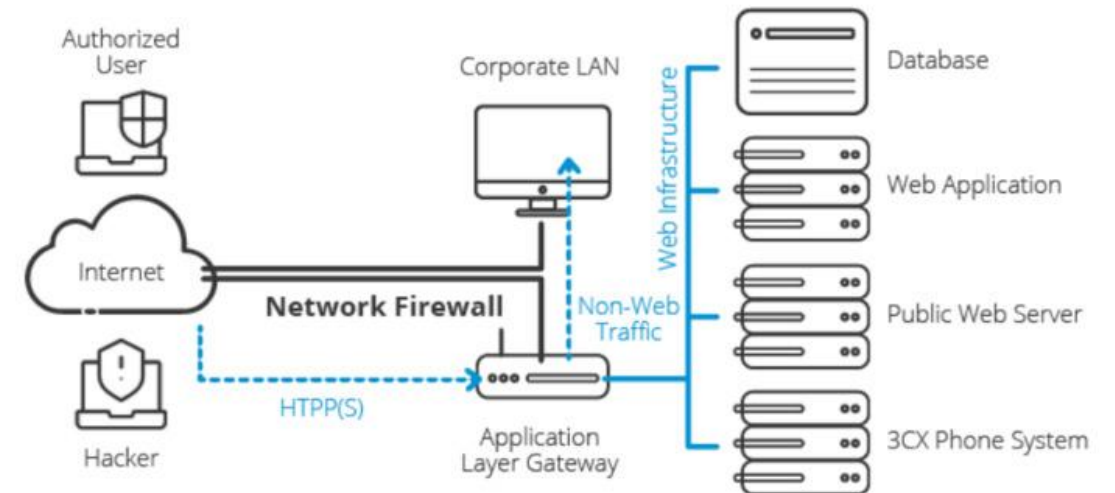
Application Gateway (Proxy Firewall) (1 of 2)

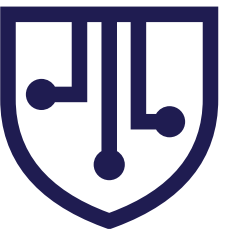
- A type of firewall that filters incoming and outgoing traffic at the **application layer (Layer 7)** by acting as an intermediary (proxy) between clients and servers.
- **How It Works:**
 - Intercepts all traffic for specific applications (e.g., HTTP, FTP)
 - Inspects and controls data based on application-layer protocols
 - Can allow, block, or modify content based on predefined rules



Application Gateway (Proxy Firewall) (2 of 2)

- **Key Features:**
 - Deep packet inspection
 - Protocol-specific filtering (e.g., only allowing FTP download, not upload)
 - User authentication and logging
 - Application-aware security policies
- **Example Protocols Supported:**
 - HTTP, HTTPS
 - FTP, SMTP, DNS
- **Use Cases:**
 - Preventing data exfiltration via email or FTP
 - Controlling access to specific web services
 - Enforcing strict policy on outbound traffic





Circuit-Level Gateway

A type of firewall that operates at the **Session Layer (Layer 5)** to monitor and manage TCP or UDP sessions between hosts

- **How It Works:**

- Establishes a virtual circuit (connection) between client and server
- Inspects the TCP/UDP handshake without inspecting the actual data
- Once the session is validated, the gateway allows traffic to pass without reinspection

- **Use Cases:**

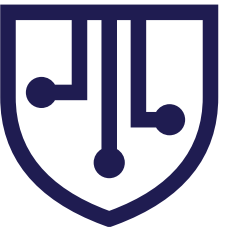
- Secure internal user access to external services
- Hiding internal IP addresses (NAT traversal)
- Performance-sensitive scenarios where full content inspection is not required

- **Key Features:**

- **Session validation** rather than content inspection
- **Fast performance** due to low processing overhead
- **Transparent operation** — clients are often unaware of the gateway
- Typically used in **SOCKS proxies**

- **Limitations:**

- Cannot detect application-layer attacks
- No payload/content analysis (e.g., cannot block SQL injection or malware)



Pretty Good Privacy (PGP) in Application Security

- PGP is an encryption program that provides cryptographic privacy and authentication for data communication.

The Four Pillars of PGP Security



Confidentiality

Ensures that only the intended recipient can read the message, keeping your data secret from unauthorized eyes.



Authentication

Verifies that the sender is who they claim to be, preventing impersonation and spoofing attacks.



Integrity

Guarantees that the message has not been altered or tampered with during transit.



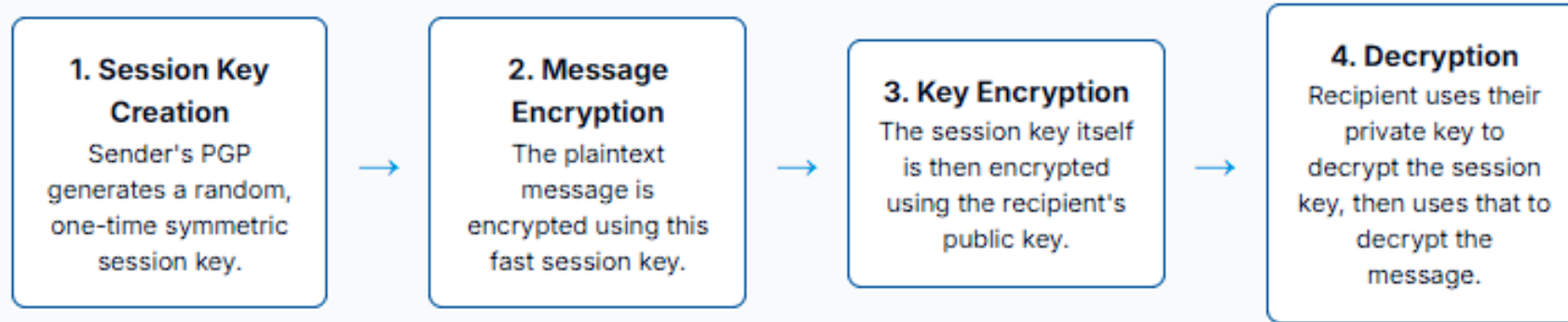
Non-Repudiation

Provides undeniable proof of authorship, preventing the sender from denying they sent the message.



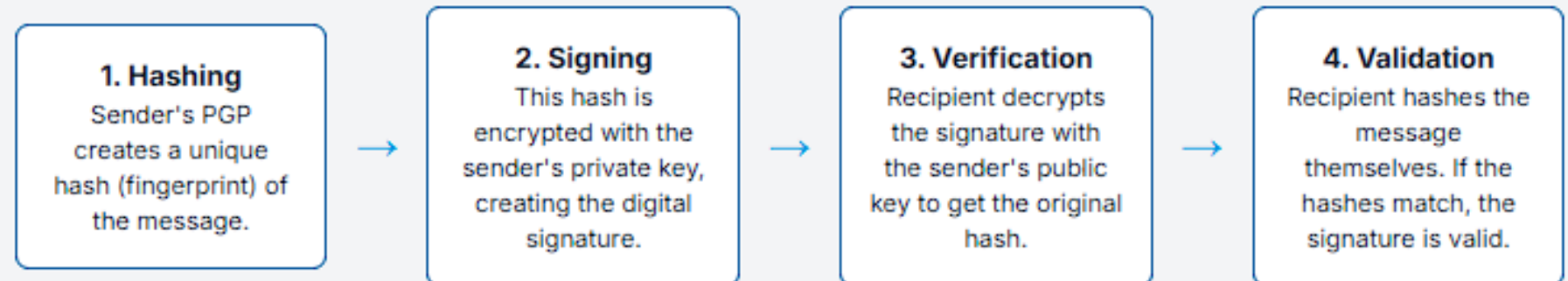
How Confidentiality Works

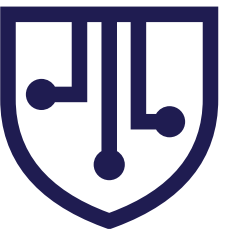
PGP uses a clever hybrid encryption system, combining the speed of symmetric encryption with the security of public-key cryptography.



How Authentication Works

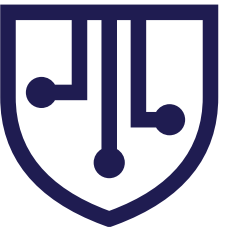
Digital signatures prove the sender's identity and the message's integrity using a process based on hashing and private keys.





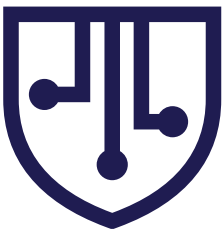
Web of Trust Concept (Decentralized Trust)

- In PGP, you **don't need a central authority** to tell you that a public key is valid.
- Instead, **you trust people you know** to vouch for the validity of others' keys.
- You can assign **trust levels** (full, marginal, none) to keys.
- This creates a **web-like structure**; trust spreads **organically**.
- Removes the need to rely on third-party Certificate Authorities (CAs) — reducing attack surfaces like CA compromise.
-  **Security Benefits:**
 - No central point of failure (unlike CAs in SSL/TLS)
 - Strong identity assurance when keys are properly verified
 - Resilient against mass surveillance or centralized control



PGP Use Cases in Application Security

1. Secure Email Communication: PGP ensures email authenticity and privacy (e.g., in Thunderbird with Enigmail)
2. Code Signing in Open Source: Developers sign software or updates (e.g., Git commits, release files)
3. File Encryption for Cloud Storage: Sensitive data can be PGP-encrypted before uploading to services like Google Drive or Dropbox
4. Secure Software Distribution: Validates that a package or app hasn't been tampered with (e.g., Debian package signing)



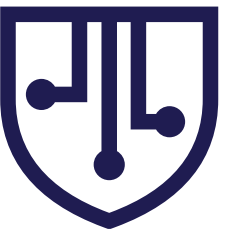
Comparison of Key Application Security Technologies

Feature	Application Gateway	Circuit-Level Gateway	Pretty Good Privacy (PGP)
Type	Application-Layer Firewall	Session-Layer Firewall	Cryptographic System
OSI Layer	Layer 7 – Application	Layer 5 – Session	Layer 7 – Application
Primary Function	Protocol-specific traffic filtering	TCP/UDP session control	Data encryption and authentication
Inspects Payload?	Yes (Deep Packet Inspection)	No	Yes (Content is encrypted/signed)
Trust Model	Centralized rule-based firewall	Centralized firewall/proxy	Decentralized (Web of Trust)
Common Protocols	HTTP, FTP, SMTP, DNS	TCP, UDP (Generic)	Email, files, messages
Main Use Cases	Content filtering, app access control	Session management, NAT traversal	Secure emails, file encryption, digital signing



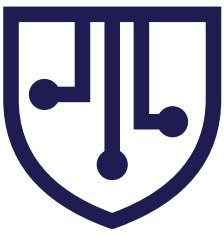
Modern Technologies/Approaches in Application Security

Technology	Purpose
WAF	Protects web apps from HTTP-based attacks
SSL/TLS	Encrypts data in transit
OAuth/OpenID/JWT	Secure identity and access
API Gateway	Secure API access and rate control
SAST/DAST/SCA	Detects code and library vulnerabilities
CSP	Restricts web resource loading
DevSecOps	Builds security into the development process
RASP	Detects and blocks attacks at runtime



Intruders in Application Security

- Intruders are unauthorized users or entities that attempt to access, modify, or disrupt an application or system.
- **Types of Intruders:**
 - **Masqueraders:** External attackers with no authorized access.
 - **Misfeasors:** Legitimate users who misuse their access.
 - **Clandestine Users:** Users who try to evade auditing and monitoring.
- **Common Intrusion Techniques:**
 - Credential theft (phishing, keylogging)
 - Session hijacking
 - Exploiting application vulnerabilities
- **Defensive Strategies:**
 - Multi-factor authentication (MFA)
 - Intrusion Detection/Prevention Systems (IDS/IPS)
 - Secure coding practices



Malicious Software in Application Security

Malware Type	Threat to Applications	Common Attack Vectors
Virus	Corrupts files, damages logic, spreads within applications	Infected executables, document macros
Worm	Resource exhaustion, remote code execution, payload delivery	Network vulnerabilities, file sharing
Trojan	Backdoors, credential theft, data exfiltration	Software downloads, email attachments
Spyware	Steals credentials, monitors input/output, privacy violation	Browser plugins, pirated apps
Adware	Degrades performance, possible delivery of other malware	Freeware bundling, misleading ads
Ransomware	Denial of application access, data loss	Phishing emails, remote desktop exploits
Rootkit	Enables persistent unauthorized access to apps and data	Privilege escalation, OS/kernel exploits
Fileless Malware	Bypasses antivirus, manipulates app processes	PowerShell, WMI, memory injection
Keylogger	Steals login credentials and sensitive input data	Browser exploits, Trojan components



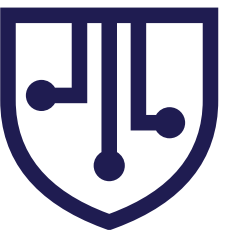
DDoS: Availability Attack to Application Security

▲ How a DDoS Attack is Performed

- **Botnet Creation:** Attackers infect many devices (PCs, IoT) with malware.
- **Command & Control:** All bots are controlled to target one server.
- **Massive Request Flood:** The botnet sends overwhelming traffic (HTTP, UDP, etc.).
- **Result:** Server crashes or becomes inaccessible to real users.
- **Why "Distributed"?** The attack comes from **many sources (by botnet)**, making it harder to block.

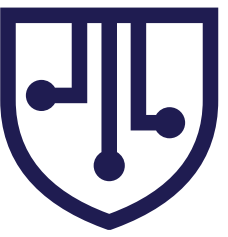
✅ Mitigation Techniques

- **Traffic Monitoring:** Detect abnormal spikes by comparing to normal patterns (benchmarking).
- **Pattern Filtering:** Identify attack signatures and block harmful packets.
- **Rate Limiting:** Restrict request volume per user/IP.
- **DDoS Protection Services:** Use platforms like Cloudflare or AWS Shield.
- **Load Balancing:** Distribute traffic across multiple servers to prevent overload.



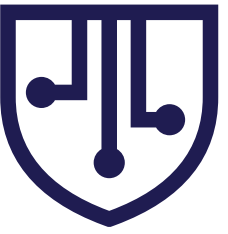
Best Practices in Application Security

-  **Apply Defense-in-Depth**
 - Layer multiple security controls across the app, network, and user levels.
 - No single solution is enough — combine prevention, detection, and response.
-  **Combine Multiple Tools and Techniques**
 - Use Application Gateways for protocol-specific filtering.
 - Implement encryption tools like PGP for data confidentiality.
 - Deploy WAFs to block web-based attacks (e.g., XSS, SQLi).
 - Pair with MFA, secure APIs, and runtime protection.



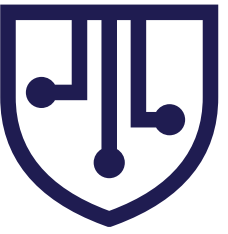
Best Practices in Application Security

-  **Keep Software and Dependencies Updated**
 - Regularly patch OS, frameworks, and libraries.
 - Use Software Composition Analysis (SCA) tools to detect known vulnerabilities.
 - Monitor CVE databases or automate with tools like Snyk, Dependabot.
-  **Conduct Regular Audits and Penetration Testing**
 - Schedule code reviews and configuration audits.
 - Perform black-box and white-box testing to identify logic flaws and implementation gaps.
 - Engage third-party or internal red teams when possible.

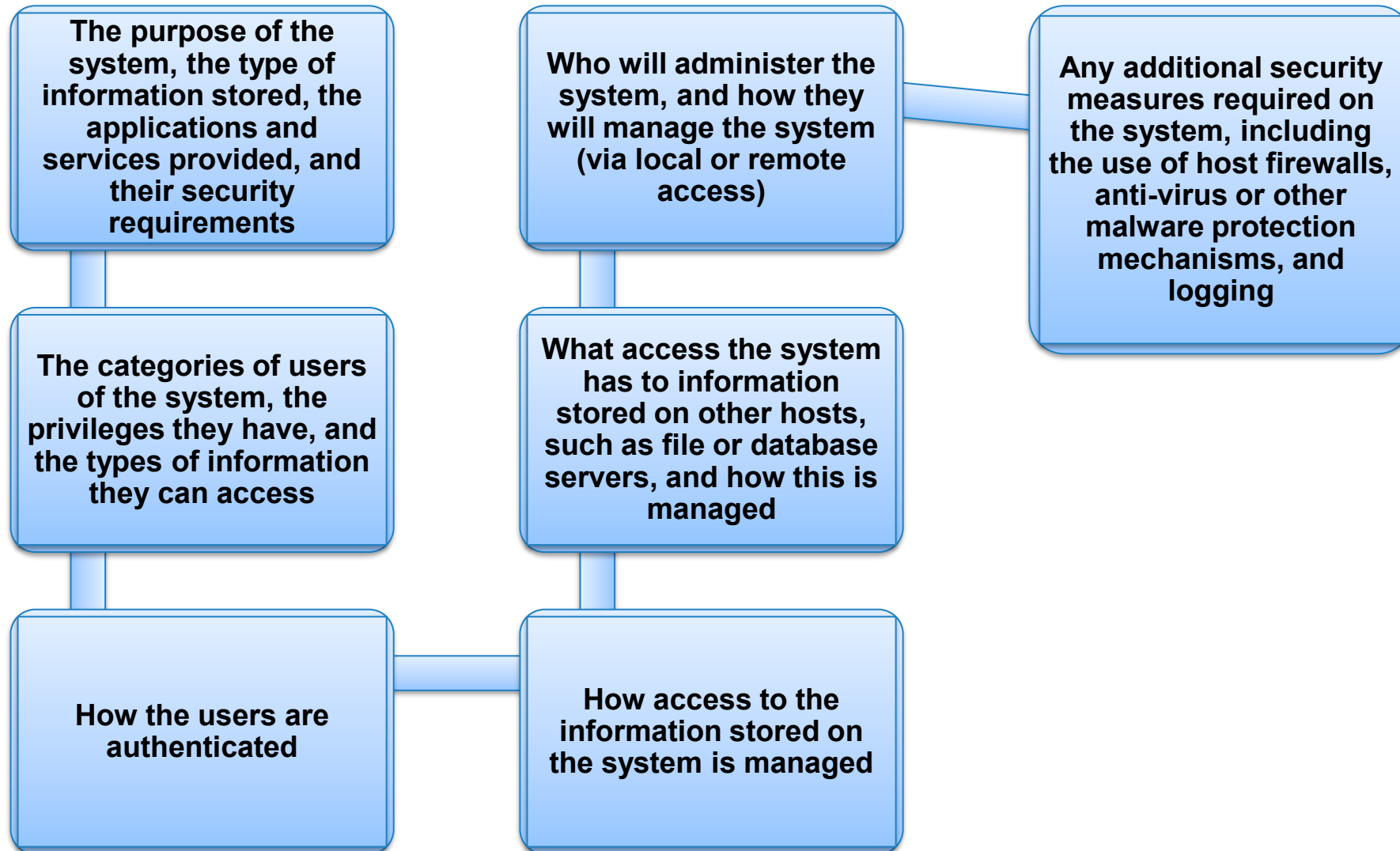


Operating System Security

- Possible for a system to be compromised during the installation process before it can install the latest patches
- Building and deploying a system should be a planned process designed to counter this threat
- Process must:
 - Assess risks and plan the system deployment
 - Secure the underlying operating system and then the key applications
 - Ensure any critical content is secured
 - Ensure appropriate network protection mechanisms are used
 - Ensure appropriate processes are used to maintain security



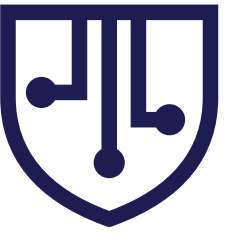
System Security Planning Process



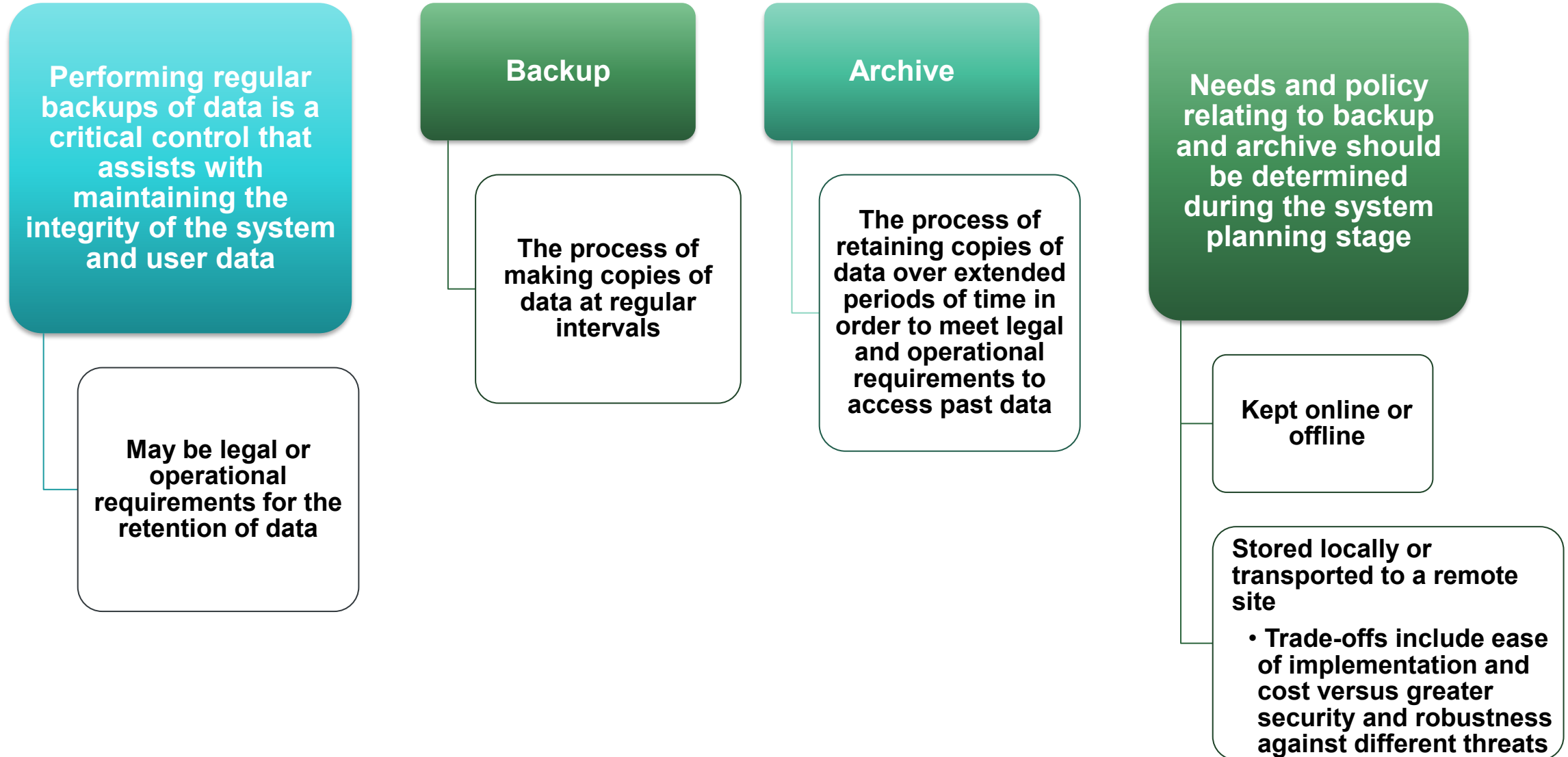


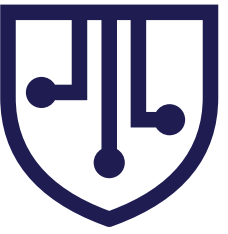
Operating System Hardening

- First critical step in securing a system is to secure the base operating system
- Basic steps
 - Install and patch the operating system
 - Harden and configure the operating system to adequately address the identified security needs of the system by:
 - Removing unnecessary services, applications, and protocols
 - Configuring users, groups, and permissions
 - Configuring resource controls
 - Install and configure additional security controls, such as anti-virus, host-based firewalls, and intrusion detection system (IDS)
 - Test the security of the basic operating system to ensure that the steps taken adequately address its security needs



Data Backup & Archive





Summary

- Application Gateway
- Circuit Level Gateway
- PGP
- Modern Technologies in Application Security
- Intruders and Malware
- Best Practices in Application Security
- Operating System Security

THANKS – Q&A



zulhilmi_abd@yahoo.com



<https://www.researchgate.net/profile/Zul-Abdullah>



<https://www.linkedin.com/in/zul-hilmi>